

Analysis: Bridge Builders

This problem seems intimidating at first glance. There are many different routes between the islands and the costs are somewhat unintuitive. However, the problem is actually a disguised Minimum Spanning Tree between the forests; and the nice thing about MSTs is that they are very friendly to greedy approaches. Almost any reasonable greedy approach that focuses on connecting up the forests will get the right answer.

It is fairly easy to convince yourself that the correct approach to the Small dataset is to head directly for the other forest; it can't possibly hurt, since all the other islands pay their minimum possible cost (ie, the distance to the nearest forest). In fact, assuming that islands pay their minimum cost is key: you can ignore this "base" cost, and then the only time you actually have to pay "extra" is when connecting up the two forests.

So for the Large dataset, once again you ignore the "base" cost for each island and simply focus on reaching all the forests, at which point you're done. Based on this intuition, you might immediately think of building a MST across the forests. The "extra" cost of connecting forests increases with distance, so use Prim's Algorithm: always head towards the forest nearest to the forests you've already visited. After connecting all the forests, you can build minimal-cost bridges to the remaining islands.

This greedy approach always works, and is quite an intuitive answer when you think of the problem in these terms. However, the proof that it works turns out to be quite technical. The problem is that you have to be sure that the forest connections really do form a graph with well-defined, order-independent costs. If your choice of path between one pair of forests could somehow help you save cost later, this would be incorrect (and the specter of NP-Completeness would loom menacingly). It seems reasonable that this can't happen, especially if you try a few examples yourself. But writing a proof in contest time is probably impossible. In fact, it took us several days of collaboration to come up with a proper, correct proof. :) This is what you have to deal with as a competitor in the Finals of the Google Code Jam!

Here's an outline of this proof:

First, suppose that we already have a tree (no cycles) of bridges which connects up every island. We show that the cheapest way we could have built these bridges is given by our MST algorithm (where distances are measured along the given tree). Assume we have some cheapest ordering of the bridges. First, we can reorder at no cost to ensure that it consists of direct sequential paths from a previously-visited island to a forest. Now suppose there is ever an island A from which we build two bridges, A-B and A-C. Further, suppose A-B leads to a closer island than A-C, but we build A-C first. Finally, suppose this is the *last* such occasion; so we build directly to the closest islands in B's subtree and in C's subtree.

Let w be the distance, when A-C is built, from A to its nearest forest along built bridges. Let x be this distance when A-B is built (so $x \leq w$). Let y be the distance from A to its closest island in B's subtree, and z to the closest island in C's subtree (so $y \leq z$). Then the cost of building these A-C and A-B paths is $(w+1 + w+2 + \dots + w+z) + (x+1 + x+2 + \dots + x+y)$. Instead, if we do the A-B path *first* and leave A-C and its subtree until later, we pay at most $(w+1 + w+2 + \dots + w+y) + (x+1 + x+2 + \dots + x+z)$, and possibly less since intermediate steps may also cost less. The second subtracted from the first is $(w+y+1)-(x+y+1) + (w+y+2)-(x+y+2) + \dots + (w+z)-(x+z) = (z-y)*(w-x)$, which is non-negative.

Thus, without increasing the cost, we can rearrange the order of bridges to build A-B first. Repeating this, we determine that one cheapest way to build the tree of bridges is always to

head for the nearest forest first, which is what our algorithm does.

Now, for any graph (a set of islands with a set of possible bridges), we prove inductively that our algorithm always gives the cheapest cost. It's true for 1 island; assume it's true for k . Suppose we have a minimal plan to connect up $k+1$ islands. Let A-B be the final bridge built. B could not have been visited already; so consider this plan restricted to the first k islands. Inductively, we know this plan is no cheaper than our algorithm, when run on those k islands; furthermore -- a nice fact about our algorithm, if you have not noticed yet -- after running our algorithm A is as close as possible to a forest. So doing this then building A-B is also a cheapest plan, and, importantly, this forms a tree of bridges. Let's call it T.

Now, from above, we know the cost of building this tree of bridges: it's just the total cost based on running our algorithm on the forest-to-forest distances in T. But these distances are at least as large as the corresponding forest-to-forest distances along *all* the potential bridges between our $k+1$ islands. We've paid at least as much as a normal spanning tree on our forests - so at least as much as a MST. But our algorithm achieves this MST cost, which we now know to be minimal.

QED.

Interestingly, this result also holds for any graph of islands and potential bridges, not just a grid or a planar graph. And the distance-to-cost metric can be any nondecreasing function. These facets of the problem are artificial and not essential to the algorithm.

Finally, here is pseudocode for the solution, assuming you know any standard MST algorithm.

```
for each unordered pair of forests (a, b)
    x = distance(a, b)
    y = x / 2 // Assume integer division
    c = x * (x + 1) / 2 // Add cost of connecting forests
    c -= y * (y + 1) // Subtract "base" cost of islands
    if x is even
        c += y // Avoid double-counting middle island
    add edge (a, b) at cost c to forest_graph

result = min_cost_spanning_tree(forest_graph)

for each island x
    d = INFINITY
    for each forest y
        d = min(d, distance(x, y))
    result += d // Add "base" cost of island x
```

Analysis: Juice

In this problem, we need to find the best (A^*, B^*, C^*) such that there is a maximum number of (A, B, C) triplets in the input satisfying:

$$A \leq A^*, B \leq B^*, C \leq C^*, \text{ and } A^* + B^* + C^* \leq 10000.$$

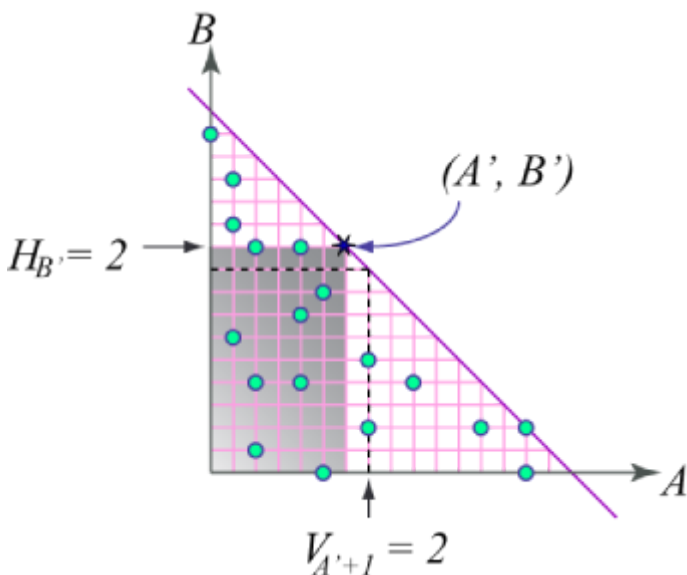
It is easy to see that we can just consider integer A^*, B^*, C^* 's. In fact C^* can be one of the C 's from the input, so are A^* and B^* -- otherwise we can just decrease it until it hits some C for a satisfied customer.

So we have at most 5000 possible candidate values for C^* (or 10000, if you don't want to use the above observation). We try each of them. And the problem is nicely visualized in 2-dimensional grid.

For a fixed C^* , we know $A^* + B^* \leq 10000 - C^*$. We filter out all the inputs that have

$$C \leq C^* \text{ or } A + B \geq 10000 - C^*,$$

and view the remaining as points in the 2-d plane, with their A, B as the coordinates. For the best solution (A^*, B^*) , we can just try all the integer points $(10001 - C^*)$ of them) on the line $A^* + B^* = 10000 - C^*$. For each point, we need to know *quickly* how many input points are dominated by that point, i.e., lying in the axis-parallel rectangle between the origin and that point.



The last step must be computed fast enough to meet the time limit of the competition. Assume we move the point (A^*, B^*) from the top-left to bottom-right. In a certain step, we are at (A', B') , with Q points dominated by it. In the next step, we are at $(A' + 1, B' - 1)$, the number of points dominated by the new point can be computed as

$$Q' = Q - H_{B'} + V_{A'+1},$$

where V_A is the counter of points on the A -th vertical line, and H_B is the number of points on the B -th horizontal line. Q' can be computed in constant time if we pre-compute the counters.

Solution from the judges:

```

int T, n, ans;
int A[5000], B[5000], C[5000], H[10001], V[10001];

int main() {
    cin>>T;
    for (int t=1; t<=T; t++) {
        cin>>n;
        for(int i=0; i<n; ++i) cin>>A[i]>>B[i]>>C[i];

        int ans = 0;
        for (int CC=0; CC<=10000; ++CC) {
            memset(H, 0, sizeof(H));
            memset(V, 0, sizeof(V));
            for (int i=0; i<n; ++i)
                if (C[i]<=CC && A[i]+B[i]+CC<=10000)
                    { V[A[i]]++; H[B[i]]++; }
            int Q = 0;
            for (int AA=-1; AA<10000-CC; ++AA) {
                Q = Q + V[AA+1] - H[10000-CC-AA];
                ans >?= Q;
            }
        }

        cout<<"Case #"<<t<<": "<<ans<<endl;
    }
    return 0;
}

```

Analysis: Mine Layer

For the convenience of discussion, we denote $c_{i,j}$ as the input number at the position (i,j) , i.e., the number of mines in the 3 by 3 square centered at (i,j) . In the solution below we will see that, any solvable square must have a unique answer for the number of mines in the middle row.

(1) The number of mines in any 3 rows and 3K columns

In the picture below, sum up the $c_{i,j}$'s of the starred positions.



(2) The number of mines in any 3 rows and C columns

If C is not a multiple of 3, based on $C \bmod 3$, use one or two squares on the boundary.

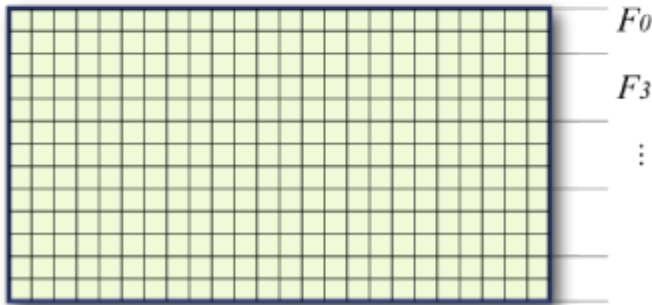
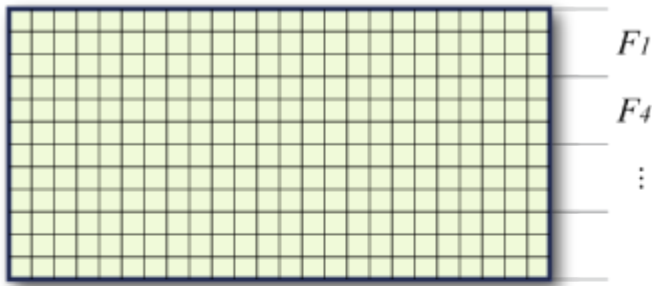


In summary, to get the number of mines in any three consecutive rows, we look at the middle row, start from either the first or second square, and mark every 3rd square. Let the middle row be the a -th row. We denote this sum by F_a .

Also note that with exactly the same method, F_0 gives the number of mines in the first two rows, and F_{R-1} does the same for the last two rows.

(3) The number of mines in the whole board

Similarly, based on $R \bmod 3$, start from $a = 0$ or 1 and sum up the F_a for every 3rd number.



(4) The number of mines in the first $3K$ or $3K+2$ rows

We omit the picture here. It is similar to the pictures above. We either group the first 2 rows together, or the first 3. By symmetry, we can also get the number of mines in the last $3K$ or $3K+2$ rows.

(5) The number of mines in the middle row

Let $h = (R - 1) / 2$. There are h rows above the middle row, as well as h rows below.

If $h \bmod 3$ is 0 or 2, we can get the sum of those $2h$ rows from step (4) and subtract it from the total number of mines from step (3).

If $h \bmod 3$ is 1, we can get the sum of the first $h+1$ rows from step (4), as well as the sum of the last $h+1$ rows. Only the center row is counted twice, so we can subtract the sum by the total number of mines from step (3).

Solution from the judges

```
int T, R, C;
int c[100][100];

int SumRowsCentered(int a) { // F(a) as in the discussion.
    int r=0;
    for(int i=(C%3)?0:1; i<C; i+=3) r+=c[a][i];
    return r;
}

int play() {
    int i;
    // Get the total.
    int total=0;
    for(i=(R%3)?0:1; i<R; i+=3) total+=SumRowsCentered(i);
    // Get the answer.
    int h=(R-1)/2; int S=0;
```

```

if (h%3==1) {
    for(i=h-1;i>=0;i-=3) S+=SumRowsCentered(i);
    for(i=h+1;i<R;i+=3) S+=SumRowsCentered(i);
    return S-total;
} else {
    for(i=h-2;i>=0;i-=3) S+=SumRowsCentered(i);
    for(i=h+2;i<R;i+=3) S+=SumRowsCentered(i);
    return total-S;
}
return 0;
}

int main() {
    int i,j,k;
    cin>>T;
    for (i=1; i<=T; i++) {
        cin>>R>>C;
        for(j=0;j<R;j++) for (k=0;k<C;k++) cin>>c[j][k];
        cout<<"Case #"<<i<<": "<<play()<<endl;
    }
    return 0;
}

```

Analysis: Ping Pong Balls

Special Cases

The small dataset can be solved by a brute force. With algorithms like breadth first search (BFS), one can find all the points that are triggered.

Another case we can use BFS is when the two displacement vectors are collinear. In this case all the points we need to consider are in a line, and there will be no more than 1000000 points on any line.

General Case

Interestingly, for the general case, where the two vectors are not collinear, one can use a shorter program with a simpler data structure. Here we introduce a solution that only involves vector additions. This is not the most efficient solution, but it is good enough for our purposes.

Note: This is not the first time we've encountered a 2-dimensional integer lattice where it is better to change the coordinate system. See the similar picture we have in the analysis for Problem D in online round 3.

Let us use $[x, y]$ to denote the points in the ordinary coordinate system, and (a, b) to denote the points in the new system. Suppose the two displacement vectors are $V_1 = [\delta_{x1}, \delta_{y1}]$, $V_2 = [\delta_{x2}, \delta_{y2}]$. Suppose that the first ball hits position $P = [x_0, y_0]$. The points in the new system is given by

$$(a, b) := P + aV_1 + bV_2 = [x_0 + a\delta_{x1} + b\delta_{x2}, y_0 + a\delta_{y1} + b\delta_{y2}]. (*)$$

The problem is to figure out, from $(0, 0)$, how many points will be hit by repeatedly adding $(1, 0)$ or $(0, 1)$.

It is not hard to prove, using $(*)$ and the fact that the two vectors are not collinear, that for any points (a, b) inside the room, the numbers a and b are bounded by some quantity Q , where Q is the size of the room times the maximum value of the δ s. With the limits in our problem, $Q \leq 2 \times 10^7$.

For any fixed a , it is easy to see that there is a contiguous sequence of numbers b such that (a, b) is hit. In other words, there are numbers b_a and b'_a such that (a, b) is hit if and only if $b_a \leq b \leq b'_a$. This is because, if we let b_a and b'_a be the minimum and maximum b that get hit, respectively, then, by convexity, (a, b) is inside the room for all b between b_a and b'_a . Once we hit (a, b_a) , we keep adding $(0, 1)$ and get all the other points. It is clear that (a, b'_a) is the last point that stays in the room, and $(a, b'_a + 1)$ is outside the room.

It is good enough if we can iterate over $a = 0, 1, 2, \dots$, (at most Q), and quickly compute b_a and b'_a for each a . Notice that, by definition, $(a, b_a - 1)$ is not hit. In order to hit (a, b_a) , the last step must be from $(a - 1, b_a)$. So

$$b_{a-1} \leq b_a \leq b'_{a-1}.$$

To find b_a , we can simply start from b_{a-1} and keep increasing until we hit a point inside the room. Note that for a single a this might take a long time. However, the b_a 's are monotone, so the cost never exceeds Q .

Once we find b_a , we can use a binary search to find b'_a . Although a binary search is simple and quick enough, another approach seems dumber but actually works faster. Similarly to the way we get b_a from b_{a-1} , we may also get b'_a from b'_{a-1} . Here the b'_a are not monotone, so we need to try both increasing and decreasing. Nevertheless, based on the rectangular shape of the room, the direction will not change more than once. The total cost is still $O(Q)$.

Sample code from the judges for the non-collinear case:

```
int T, W, H;
int x, y, dx1, dx2, dy1, dy2;

bool inside(int a, int b) {
    int xx = x + a*dx1 + b*dx2;
    int yy = y + a*dy1 + b*dy2;
    if(xx<0 || xx>=W) return false;
    if(yy<0 || yy>=H) return false;
    return true;
}

long long play() {
    long long ans=0;
    int b1=0; int b2=1000001;
    for(int a=0; ; a++) {
        while(!inside(a, b1)) {
            b1++;
            if(b1>b2) return ans;
        }
        if(inside(a, b2)) {
            while(inside(a, b2)) b2++;
            b2--;
        } else {
            while(!inside(a, b2)) b2--;
        }
        ans+=(b2-b1+1);
    }
    return 0;
}
```

Analysis: The Year of Code Jam

The different appearances of perimeter

In spite of the nice story about the calendar and the fancy definition of happiness, we hope that you have discovered the following abstract description of this problem.

On a board of N by M unit squares, some squares are blue, some are white, and some are undecided. For each undecided square, color it either blue or white, so that the total perimeter of blue region is maximized.

We see that the perimeter can be defined in two ways.

- (a) The sum of the contributions from each blue square, as indicated in our problem statement.
- (b) The sum of the contributions from each unit segment. Count 1 for each unit segment that separates two squares with different colors. (Assume all the squares outside the board are white.)

The second interpretation is useful for the following analysis.

There are some simple observations that seem useful. For example, for any undecided unit square, if we have already decided that two of its neighbors are blue, then in one optimal solution we can assign it to be white. Yet, as far as we know, no such heuristics give a complete and fast solution to our problem.

A similar problem

Let us discuss a problem that at least looks similar to ours. What if we wanted to minimize the perimeter instead of maximizing it?

We rephrase the problem in terms graph theory. Let the set of NM unit squares be our vertices, and let there be an edge between two adjacent squares. Our task is to color each undecided vertex either blue or white, so that the number of edges (in the graph) between the blue vertices and the white vertices is minimized.

This is rather nice, isn't it? If you add a source s and a destination t , add one edge from s to every blue vertex with infinite capacity (4 is enough) and one edge from every white vertex to t with infinite capacity, then the problem is asking for a *minimum cut* from s to t , which can be solved by your favorite max flow algorithm.

Solving the original problem

While minimum cut is polynomial-time solvable by max flow, the max-cut problem is too hard in general graphs. Therefore, our problem still seems much harder than the minimization version of the problem.

However, we have not used an important fact yet. Our graph is far from arbitrary. We play this game on the N by M board; the resulting graph is bipartite (imagine it as a chessboard). Maximizing the number of edges between different colored vertices is the same as minimizing the number of edges between vertices of the same color. In other words, if we flip the colors of

one half of the chessboard, the problem reduces to the minimization version we discussed above.

We mark the board in a chessboard fashion, label the unit squares as odd or even. Then we flip the color of all the even squares. Now look at property (b). A contribution becomes a non-contribution and vice versa. The total number of edges is fixed, so the maximization problem reduces to its minimization version.